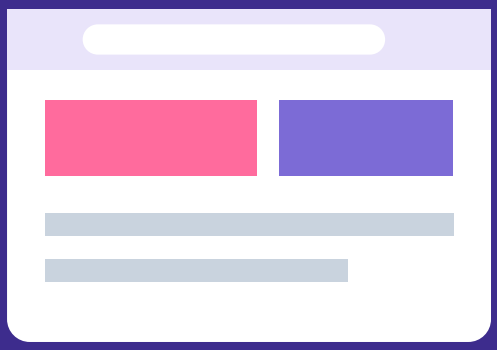

Frontend Development Starter Guide

HTML, CSS, JavaScript, frameworks and modern tooling

Structure • Style • Interact • Ship

Updated October 2026

Contents

1. What frontend development is
2. HTML: structure
3. CSS: style and layout
4. JavaScript: behavior
5. Tooling and TypeScript
6. Frameworks and libraries
7. Accessibility and performance
8. Testing and deployment
9. Roadmap and project ideas

> **Who is this for?**
> Beginners who want to build websites and web apps, and developers moving from other areas. You only need a computer, a browser and a text editor such as VS Code.

The three core languages

Language	Role	Analogy
HTML	Defines the content and structure	The skeleton
CSS	Controls appearance and layout	The skin and clothes
JavaScript	Adds behavior and interactivity	The muscles and nerves

1. What frontend development is

The **frontend** is everything users see and interact with in a browser: layouts, buttons, forms, animations and the data displayed on screen. It talks to a **backend** (servers and databases) through APIs.

What frontend developers do

- Turn designs (often from Figma) into working pages.
- Make sites responsive on phones, tablets and desktops.
- Fetch and display data from APIs, and handle user input.
- Ensure pages are fast, accessible and work across browsers.
- Write tests and maintain a clean, reusable component library.

How a web page loads

- The browser requests a URL and receives an **HTML** file.
- It finds linked **CSS** and **JavaScript** files and downloads them.
- It builds the **DOM** (a tree of page elements) and applies styles.
- JavaScript runs, updates the DOM and calls APIs when needed.
- The browser paints the result on screen.

> **Try it now**
> Open any website, press **F12** to open DevTools, and inspect the HTML and CSS. It is the fastest way to learn how real sites are built.

2. HTML: structure

HTML uses **elements** written as tags to describe content. Use **semantic** elements that describe meaning, not just appearance; they help accessibility and search engines.

A minimal page

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My First Page</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header><h1>Welcome</h1></header>
  <main>
    <p>Hello, <a href="/about">learn more</a>.</p>
    <img src="photo.jpg" alt="A mountain at sunrise">
  </main>
  <footer>&copy; 2026</footer>
  <script src="app.js" defer></script>
</body>
</html>
```

Elements to know

Group	Elements
Page structure	`header, nav, main, section, article, aside, footer`
Text	`h1-h6, p, strong, em, ul, ol, li, blockquote, code`
Links and media	`a, img, video, audio, picture`
Forms	`form, label, input, select, textarea, button`
Tables	`table, thead, tbody, tr, th, td`

Always label inputs
Connect each form field to a label with `<label for="email">` and a matching `id`. It improves usability and accessibility.

3. CSS: style and layout

CSS selects elements and applies rules. Learn four ideas first: the **box model**, **Flexbox**, **Grid** and **responsive design**.

The box model

Every element is a box: **content**, surrounded by **padding**, then **border**, then **margin**. Set `box-sizing: border-box` so width includes padding and border.

```
:root {
  --brand: #3d2c8d;
  --radius: 8px;
}
* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  line-height: 1.5;
}
.button {
  background: var(--brand);
  color: white;
  padding: 0.6rem 1.2rem;
  border-radius: var(--radius);
}
.button:hover { opacity: 0.9; }
```

Flexbox vs Grid

Layout tool	Use it for	Key properties
Flexbox	One-dimensional layouts: a row or a column (navbars, button groups, centering)	display:flex; justify-content; align-items; gap
Grid	Two-dimensional layouts: rows and columns together (page layouts, card galleries)	display:grid; grid-template-columns; gap

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1rem;
}
```

Responsive design

Design **mobile-first**: write styles for small screens, then add **media queries** to enhance larger ones.

```
.layout { display: block; }

@media (min-width: 768px) {
  .layout {
    display: grid;
    grid-template-columns: 240px 1fr;
  }
}
```

CSS tips and tools

- Use **rem** and **%** units rather than fixed pixels for flexible layouts.
- Use **CSS variables** (custom properties) for colors, spacing and themes; they also make dark mode easy.
- Use `clamp()` for fluid font sizes, for example `font-size: clamp(1rem, 2vw, 1.5rem)`.
- Keep selectors simple and avoid `!important`.

Styling approaches

Approach	Description
Plain CSS	Best for learning; modern CSS is very capable.
CSS Modules	Locally scoped class names per component.
Tailwind CSS	Utility classes written directly in HTML or JSX, such as `p-4 text-lg`.
Sass / SCSS	CSS with nesting and mixins; less necessary now that CSS has variables and nesting.
Component libraries	Ready-made components: Material UI, Chakra UI, shadcn/ui, Bootstrap.

4. JavaScript: behavior

JavaScript runs in the browser and makes pages interactive. Learn the language basics first: variables, functions, arrays, objects, and modules.

Core syntax

```
const name = "Ada";              // value that cannot be reassigned
let count = 0;                   // value that can change

const double = (n) => n * 2;     // arrow function

const user = { name: "Ada", age: 36 };
const { name: userName } = user; // destructuring

const nums = [1, 2, 3];
const doubled = nums.map(double);        // [2, 4, 6]
const evens = nums.filter(n => n % 2 === 0);

console.log(`Hello, ${userName}!`);
```

Working with the DOM

```
const button = document.querySelector("#add");
const list = document.querySelector("#todos");

button.addEventListener("click", () => {
  const li = document.createElement("li");
  li.textContent = "New task";
  list.append(li);
});
```

Fetching data from an API

Use `fetch` with `async/await`, and always handle errors.

```
async function loadUsers() {
  try {
    const res = await fetch("https://api.example.com/users");
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const users = await res.json();
    console.log(users);
  } catch (err) {
    console.error("Could not load users:", err);
  }
}
```

5. Tooling and TypeScript

Modern frontend projects use a build pipeline to bundle code, run a dev server with instant reload and optimize output for production.

Essential tools

Tool	Purpose
Node.js and npm	Runs JavaScript tools and installs packages (pnpm and yarn are alternatives).
Vite	Fast dev server and bundler; the common starting point for new projects.
TypeScript	JavaScript with types, catching many errors before you run the code.
ESLint	Finds problems and enforces coding rules.
Prettier	Formats code automatically for a consistent style.
Browser DevTools	Inspect elements, debug scripts, check network and performance.
Git	Version control for every project.

Create a project with Vite

```
npm create vite@latest my-app
cd my-app
npm install
npm run dev        # start the dev server
npm run build      # production build in dist/
```

TypeScript in one example

```
interface User {
  id: number;
  name: string;
  email?: string;      // optional
}

function greet(user: User): string {
  return `Hello, ${user.name}`;
}

greet({ id: 1, name: "Ada" });   // OK
greet({ id: 2 });                // Error: name is missing
```

Learn JavaScript first
TypeScript builds on JavaScript. Get comfortable with plain JavaScript, then adopt TypeScript. Most professional projects use it.

6. Frameworks and libraries

Frameworks help you build large interfaces from reusable **components** and keep the screen in sync with your data (**state**).

Framework	Style	Notes
React	Component library using JSX	Largest ecosystem and job market; Next.js and Remix add routing and server rendering.
Vue	Progressive framework with single-file components	Gentle learning curve; Nuxt adds full-stack features.
Angular	Complete opinionated framework (TypeScript)	Batteries included; popular in large enterprises.
Svelte	Compiler-based, minimal boilerplate	Small bundles, easy syntax; SvelteKit for apps.
Astro	Content-focused sites with little JavaScript	Great for blogs, docs and marketing sites.

A React component

```
import { useState } from "react";

export function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Key concepts in every framework

- **Components**: small reusable pieces of UI that accept inputs (props).
- **State**: data that changes over time and drives what is displayed.
- **Routing**: showing different views for different URLs.
- **Data fetching**: loading and caching server data (for example TanStack Query).
- **Global state**: sharing data across components (Context, Redux Toolkit, Zustand, Pinia).

Which should I pick?
For jobs and community support, start with **React**. For an easier first experience, try **Vue** or **Svelte**. For content sites, consider **Astro**. Concepts transfer between all of them.

7. Accessibility and performance

Accessibility (a11y)

Accessible sites work for everyone, including people using screen readers, keyboards or magnification. It is also a legal requirement in many places. The standard is **WCAG**.

- Use **semantic HTML** (real buttons, links, headings) before reaching for ARIA attributes.
- Provide **alt text** for meaningful images; use empty `alt=""` for decorative ones.
- Make everything usable by **keyboard**, with a visible focus outline.
- Keep **color contrast** high (at least 4.5:1 for normal text).
- Do not rely on color alone to convey information.
- Test with a screen reader and tools such as Lighthouse and axe DevTools.

Performance

Fast pages keep users and improve search ranking. Google measures **Core Web Vitals**:

Metric	Measures	Good target
LCP (Largest Contentful Paint)	How quickly the main content appears	2.5 s or less
INP (Interaction to Next Paint)	How fast the page responds to input	200 ms or less
CLS (Cumulative Layout Shift)	How much the layout jumps around	0.1 or less

Performance habits

- Compress and size images; use modern formats (WebP, AVIF) and `loading="lazy"`.
- Ship less JavaScript: split code, remove unused packages.
- Set image width and height to avoid layout shifts.
- Cache static assets and serve them from a CDN.
- Measure with Lighthouse and the browser Performance tab before optimizing.

8. Testing and deployment

Testing

Kind	Tools	What it checks
Unit	Vitest, Jest	Individual functions and logic
Component	Testing Library	Components render and respond to user actions
End-to-end	Playwright, Cypress	Complete user journeys in a real browser
Visual	Storybook, Chromatic	Appearance of components

Deployment

Build your project, then host the generated files. Static hosts with Git integration redeploy automatically on every push.

Host	Good for
Netlify, Vercel, Cloudflare Pages	Static sites and frameworks; free tiers, preview URLs for pull requests
GitHub Pages	Simple static sites straight from a repository
Firebase Hosting, AWS Amplify, Azure Static Web Apps	Hosting tied to a larger cloud platform

Environment and security basics

- Anything shipped to the browser is **public**. Never put secret keys in frontend code; call a backend instead.
- Escape or sanitize user content to prevent **XSS**; frameworks do much of this by default.
- Serve everything over **HTTPS**.
- Keep dependencies updated and run `npm audit`.

Git to live in four steps

- Push your project to GitHub.
- Connect the repository to Netlify, Vercel or Cloudflare Pages.
- Set the build command (`npm run build`) and output folder (`dist`).
- Every push now deploys automatically.

9. Roadmap and project ideas

Suggested learning path

Stage	Learn	Build
1. Foundations (4-6 weeks)	HTML, CSS, Flexbox, Grid, responsive design, Git	Personal portfolio, landing page
2. JavaScript (6-8 weeks)	Language basics, DOM, events, fetch, async/await	To-do list, weather app, quiz
3. Tooling (2 weeks)	npm, Vite, ES modules, ESLint, Prettier, TypeScript basics	Rebuild an earlier project with Vite
4. A framework (6-8 weeks)	Components, state, routing, forms, data fetching	Movie search, expense tracker
5. Professional skills (ongoing)	Testing, accessibility, performance, CI, design systems	Full app with login, API and tests

Project ideas

- **Beginner:** portfolio page, tip calculator, image gallery, responsive navbar.
- **Intermediate:** weather dashboard (public API), Pomodoro timer, markdown previewer, quiz app.
- **Advanced:** e-commerce front end with cart, Kanban board with drag and drop, real-time chat UI.

Free learning resources

- **MDN Web Docs** (developer.mozilla.org): the best reference for HTML, CSS and JavaScript.
- **web.dev** (Google): guides on performance, accessibility and modern CSS.
- **javascript.info**: a thorough free JavaScript tutorial.
- **freeCodeCamp** and **The Odin Project**: free structured curriculums.
- **react.dev, vuejs.org, svelte.dev**: official framework tutorials.

> **Final advice**
> Build something real at every stage. Reading is not enough: break things, fix them, and publish your work. A small project you finish teaches more than ten tutorials you only watch.